BÀI THỰC HÀNH SỐ 5: CASE STUDY CÀI ĐẶT KIẾN TRÚC MICROSERVICES TRÊN MÔI TRƯỜNG .NET CORE (BÀI TOÁN ĐẠI LÝ BẢO HIỂM)

Phần 3: DỊCH VỤ DISCOVERY VÀ CÀI ĐẶT EUREKA

Tiếp theo bài thực hành số 4, sinh viên thực hiện các task tiếp theo như sau:

Task #1: Bổ sung code cho microservice PricingService

- Copy code từ các file vào project. Trường hợp không add vào được thì tạo thủ công:
    - Tạo các file trùng tên
    - Copy code/nội dung của file vào project
- Lấy code ở file nén đính kèm.
- Trường hợp tên projects hoặc tên các file có khác nhau thì xử lý theo 1 trong 2 phương án:
    - Đổi lại tên cho phù hợp với code mẫu thầy gửi (*khuyên nên dùng phương án này*)
    - Giữ nguyên tên theo project của sinh viên và chú ý đổi tên ở tất cả những nơi mà có được sử dụng.

Task #2: Cài đặt và cấu hình liên quan đến EUREKA

- Các dịch vụ cần giao tiếp với nhau.
- Muốn giao tiếp thì Dịch vụ gọi phải biết chính xác vị trí của microservice mục tiêu (được gọi) trong mạng mà chúng đang hoạt động.
- Dịch vụ mục tiêu phải cung cấp địa chỉ IP và cổng (port) mà mình đang lắng nghe.
- Trước đây có nhiều kỹ thuật để giúp các microservice giao tiếp với nhau nhưng theo cách thủ công và dữ liệu đăng ký đăng ký ở trạng thái tĩnh.
- Dịch vụ Khám phá là một cơ chế trong kiến trúc Microservice cho phép các dịch vụ tìm thấy vị trí mạng của nhau một cách tự động và dữ liệu đăng ký ở cơ chế động.
- Tìm hiểu về Service Registry component và Service Registry Clients.
- Service Registry là một thành phần trung tâm duy trì danh sách các phiên bản microservice hiện đang chạy cùng với vị trí mạng tương ứng của chúng. Service Registry Clients được các microservice sử dụng để: tự đăng ký vào registry, truy vấn registry để lấy địa chỉ của một microservice cụ thể mà chúng cần liên lạc.

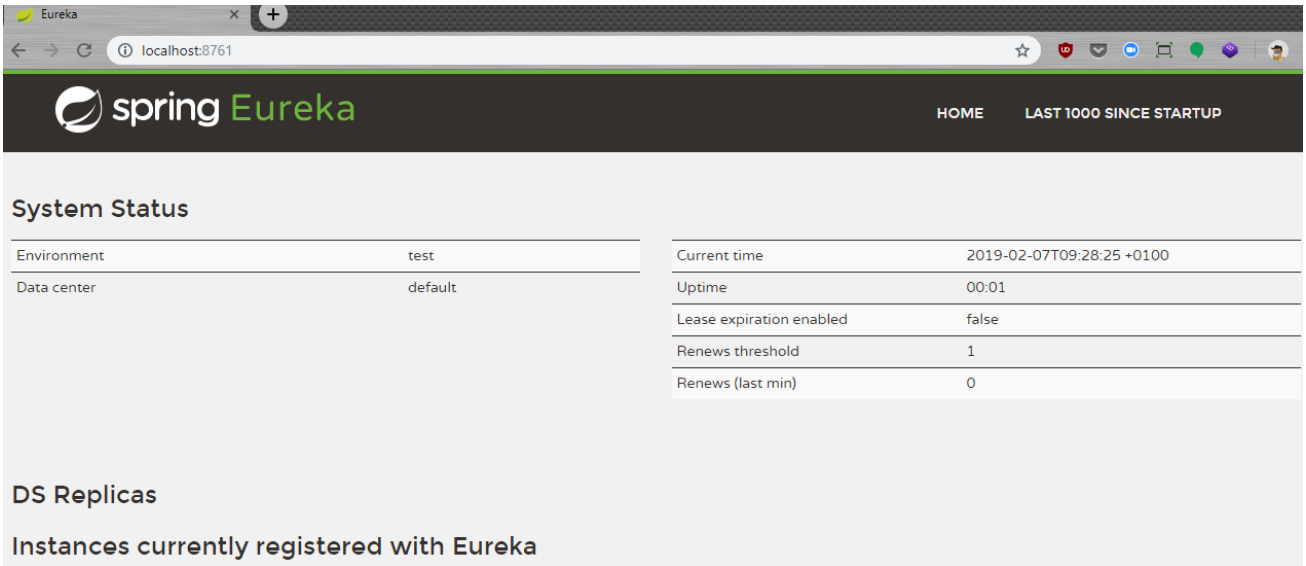
- Chúng ta sử dụng Eureka vì nó có thư viện client rất tốt dành cho .NET Core

Nhiệm vụ cụ thể

1) Cấu hình máy chủ Eureka được lưu trữ trong tệp application.yml. Hãy tìm đến tập tin này xem đã có file này trong hệ thống thư mục chưa? Thông tin ban đầu trong file:

```
server:
  port: 8761
eureka:
  client:
    registerWithEureka: false
    fetchRegistry: false
  server:
    waitTimeInMsWhenSyncEmpty: 0
```

2) Thực hiện lệnh chạy trên trình duyệt: localhost:8761 để kiểm tra xem đã cài đặt Eureka chưa, nếu chưa thì cài đặt Eureka. Trường hợp đã có thì sẽ xuất hiện trang thông tin tương tự như sau:



3) Đăng ký microservice trong Eureka / Đăng ký Client Eureka

Bước 1 – Thêm các gói cần thiết

- Steeltoe.Discovery.ClientCore ///Eureka của Steeltoe: kiểm tra gói này đã có trong code chưa, nếu chưa import vào.

Bước 2 – thêm ứng dụng khách khám phá

- Tìm đến file Startup.cs
- Tìm đến 2 phương thức: ConfigureServices và Configure.
- Xem đã có các dòng lệnh như dưới đây chưa, nếu chưa thì thêm vào

```
using Steeltoe.Discovery.Client;

namespace PricingService
{
    public class Startup
    {
        public Startup(IConfiguration configuration)
        {
            Configuration = configuration;
        }

        public IConfiguration Configuration { get; }

        public void ConfigureServices(IServiceCollection
services)
        {
            services.AddDiscoveryClient(Configuration);
            [...]
        }

        public void Configure(IApplicationBuilder app,
IHostingEnvironment env)
        {
           [...]
            app.UseDiscoveryClient();
        }
    }
}
```

Bước 3 – Cấu hình

- Thiết lập cấu hình trong tệp appsettings.json

```
"spring" : {
    "application" : {
      "name" : "PricingService"
    }
  },
  "eureka" : {
    "client" : {
```

```
      "shouldRegisterWithEureka" : true,
      "serviceUrl" : "http://localhost:8761/eureka",
      "ValidateCertificates":  false
    },
    "instance" : {
      "appName" : "PricingService",
      "hostName" : "localhost",
      "port" : "5040”
    }
  }
```

Bước 4 – Kiểm thử

- Từ thư mục gốc của kho lưu trữ: thực thi dòng lệnh: dotnet run --project ./PricingService
- Mở Eureka để xem dịch vụ có hiển thị hay không. Mở trình duyệt và truy cập localhost:8761

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
PRICINGSERVICE	n/a (1)	(1)	UP (1) -

Nếu mọi thứ hoạt động như mong đợi, một phiên bản của dịch vụ sẽ được liệt kê trong phần “Các phiên bản hiện đang được đăng ký với Eureka”.

Task #3:

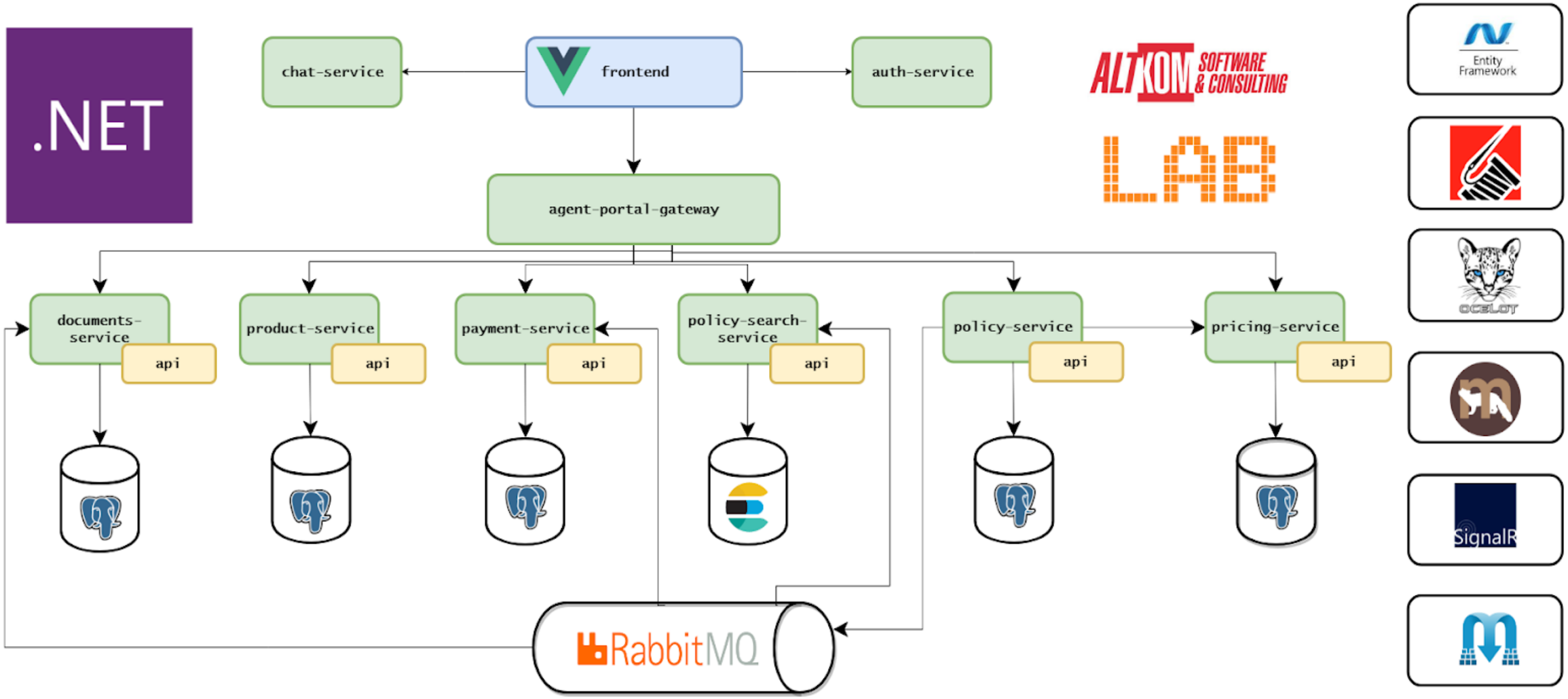
Còn tiếp…

Thời hạn nộp: chậm nhất 18h00 hôm nay (ngày 10/03/2026).

./.

PHỤ LỤC:

KIẾN TRÚC THAM KHẢO THEO YÊU CẦU CỦA BÀI THỰC HÀNH SỐ 3



Hệ thống bao gồm:

1) **Giao diện người dùng (Frontend)** – một ứng dụng trang đơn (Single Page Application) sử dụng VueJS cung cấp cho các đại lý bảo hiểm khả năng lựa chọn sản phẩm phù hợp cho khách hàng, tính toán giá cả, tạo báo giá và hoàn tất quy trình bán hàng bằng cách chuyển đổi báo giá thành hợp đồng bảo hiểm. Ứng dụng này cũng cung cấp các chức năng tìm kiếm và xem thông tin về hợp đồng và báo giá. Giao diện người dùng giao tiếp với các dịch vụ phụ trợ thông qua cổng API.
2) **Cổng API Đại lý (Agent Portal API Gateway)** – là một microservice đặc biệt với mục đích chính là che giấu sự phức tạp của cấu trúc dịch vụ văn phòng phụ trợ khỏi ứng dụng khách. Thông thường, chúng ta tạo một cổng API riêng cho mỗi ứng dụng khách. Trong trường hợp trong tương lai chúng ta thêm ứng dụng di động Xamarin vào hệ thống, chúng ta sẽ cần xây dựng một cổng API riêng cho nó. Cổng API cũng cung cấp rào cản bảo mật và không cho phép các yêu cầu chưa được xác thực được chuyển đến các dịch vụ phụ trợ. Một ứng dụng phổ biến khác của cổng API là tổng hợp nội dung từ nhiều dịch vụ.

3) **Dịch vụ xác thực (Auth Service)** – một dịch vụ chịu trách nhiệm xác thực người dùng. Hệ thống bảo mật của chúng ta sẽ dựa trên mã thông báo JWT. Sau khi người dùng xác thực chính xác, dịch vụ xác thực sẽ cấp một mã thông báo được sử dụng để kiểm tra quyền của người dùng và các sản phẩm có sẵn.
4) **Dịch vụ Chat (Chat Service)** – một dịch vụ sử dụng SignalR để cho phép các nhân viên hỗ trợ trò chuyện với nhau.
5) **Dịch vụ Định giá (Pricing Service)** – một dịch vụ chịu trách nhiệm tính toán giá cho sản phẩm bảo hiểm cụ thể dựa trên các tham số của nó.
6) **Dịch vụ Chính sách (Policy Service)** – dịch vụ chính trong hệ thống của chúng ta. Nó chịu trách nhiệm tạo ra các gói bảo hiểm và chính sách. Nó sử dụng Dịch vụ Định giá thông qua các cuộc gọi HTTP REST để tính toán giá. Sau khi một chính sách được tạo, nó sẽ gửi sự kiện không đồng bộ đến bus sự kiện (RabbitMQ) để các dịch vụ khác có thể phản hồi.
7) **Dịch vụ Tìm kiếm Chính sách (Policy Search Service)** – mục đích duy nhất của dịch vụ này là cung cấp chức năng tìm kiếm. Dịch vụ này đăng ký các sự kiện liên quan đến vòng đời của chính sách và lập chỉ mục chính sách đã cho trong ElasticSearch để cung cấp khả năng tìm kiếm nâng cao.
8) **Dịch vụ Thanh toán (Payment Service)** – dịch vụ này chịu trách nhiệm quản lý các hoạt động tài chính liên quan đến chính sách. Nó đăng ký các sự kiện liên quan đến chính sách, tạo tài khoản chính sách khi chính sách được tạo, đăng ký các khoản thanh toán dự kiến. Nó cũng có một tác vụ nền rất đơn giản để phân tích tệp chứa các khoản thanh toán đến và gán chúng vào tài khoản chính sách phù hợp.
9) **Dịch vụ Sản phẩm (Product Service)** – đây là danh mục sản phẩm. Nó cung cấp thông tin cơ bản về từng sản phẩm bảo hiểm và các thông số có thể tùy chỉnh khi tạo báo giá cho khách hàng.
10) **Dịch vụ Tài liệu (Document Service)** – dịch vụ này sử dụng JS Report để tạo chứng chỉ PDF.

Mỗi microservice được xây dựng xung quanh ngữ cảnh giới hạn (bounded context) theo thuật ngữ DDD. Bạn cũng có thể thấy rằng chúng hợp tác để đạt được mục tiêu kinh doanh chính – bán sản phẩm bảo hiểm cho khách hàng cuối cùng.